🔁 CATEGORY 5: Meta-Arguments & Lifecycle

> **Difficulty:** Intermediate | **Topics:** 6 | **Terraform Interview Mastery Series**

Table of Contents

1. `count` — Indexed Resource Creation, Limitations
2. ⚠ `for_each` — Map/Set Based, Why It's Better Than count
3. ⚠ `count` vs `for_each` — The Critical Difference
4. `lifecycle` Block — `create_before_destroy`, `prevent_destroy`, `ignore_changes`
5. `provider` Meta-Argument — Cross-Account, Cross-Region Patterns
6. ⚠ `replace_triggered_by` — When and Why (Terraform 1.2+)

Topic 29: `count` — Indexed Resource Creation, Limitations

🔵 What It Is (Simple Terms)

`count` is a meta-argument that creates **multiple instances of a resource** using integer-based indexing. Instead of writing three identical resource blocks, you write one with `count = 3` and Terraform creates three.

🔵 Why It Exists — What Problem It Solves

Without `count`, creating N identical or similar resources requires N identical blocks — pure repetition:

```
# ❌ Without count — copy-paste for every instance
resource "aws_instance" "web_1" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.medium"
}
resource "aws_instance" "web_2" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.medium"
}
resource "aws_instance" "web_3" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.medium"
}
```

With `count`:

```
# ✅ With count — one block, three instances
resource "aws_instance" "web" {
  count         = 3
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.medium"

  tags = {
    Name = "web-${count.index}"   # count.index = 0, 1, 2
  }
}
```

🔵 How `count` Works Internally

```
count = 3 creates three state entries:
  aws_instance.web[0]    → i-aaa
  aws_instance.web[1]    → i-bbb
  aws_instance.web[2]    → i-ccc

Each instance is independently managed in state
count.index provides the current index (0-based) within the block
```

🔵 Full `count` Syntax and Patterns

Basic Count

```
resource "aws_subnet" "private" {
  count             = 3
  vpc_id            = aws_vpc.main.id
  cidr_block        = cidrsubnet(var.vpc_cidr, 8, count.index)
  availability_zone =
data.aws_availability_zones.available.names[count.index]

  tags = {
    Name = "private-subnet-${count.index + 1}"   # 1-based for human
readability
  }
}

# Reference specific instance:
aws_subnet.private[0].id    # first subnet
aws_subnet.private[1].id    # second subnet

# Reference all instances (splat):
aws_subnet.private[*].id    # ["subnet-1", "subnet-2", "subnet-3"]
```

Conditional Resource Creation (count = 0 or 1)

```
# The most common count pattern in production
resource "aws_eip" "nat" {
  count  = var.enable_nat_gateway ? 1 : 0    # create or don't
  domain = "vpc"
}

resource "aws_cloudwatch_log_group" "app" {
  count             = var.enable_logging ? 1 : 0
  name              = "/app/${var.environment}"
  retention_in_days = 30
}

# Reference with index when using count = 0 or 1:
nat_gateway_id = var.enable_nat_gateway ? aws_eip.nat[0].id : null
```

Count from Variable

```
variable "instance_count" {
  type    = number
  default = 2
}

resource "aws_instance" "app" {
  count         = var.instance_count
  ami           = data.aws_ami.ubuntu.id
  instance_type = var.instance_type

  tags = {
    Name  = "${var.prefix}-app-${count.index}"
    Index = count.index
  }
}
```

Count with List Variable

```
variable "availability_zones" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1b", "us-east-1c"]
}

resource "aws_subnet" "private" {
  count             = length(var.availability_zones)
  vpc_id            = aws_vpc.main.id
  cidr_block        = cidrsubnet(var.vpc_cidr, 8, count.index)
  availability_zone = var.availability_zones[count.index]
```

```
  tags = {
    Name = "private-${var.availability_zones[count.index]}"
  }
}
```

🔵 Accessing Count-Based Resources

```
# In other resources — reference by index
resource "aws_lb_target_group_attachment" "web" {
  count            = var.instance_count
  target_group_arn = aws_lb_target_group.web.arn
  target_id        = aws_instance.web[count.index].id   # same index
  port             = 80
}

# In outputs
output "instance_ids" {
  value = aws_instance.web[*].id                        # splat — all IDs
}

output "first_instance_ip" {
  value = aws_instance.web[0].public_ip                 # specific index
}

output "instance_ips" {
  value = [for instance in aws_instance.web : instance.public_ip]
}
```

🔵 The Core Limitation of `count` — Index Instability

This is the most important thing to understand about `count` — and what makes it dangerous for non-uniform resources.

```
variable "availability_zones" {
  default = ["us-east-1a", "us-east-1b", "us-east-1c"]
}

resource "aws_subnet" "private" {
  count             = length(var.availability_zones)
  availability_zone = var.availability_zones[count.index]
}

# Current state:
# aws_subnet.private[0] → us-east-1a → subnet-aaa
# aws_subnet.private[1] → us-east-1b → subnet-bbb
# aws_subnet.private[2] → us-east-1c → subnet-ccc
```

```
# Now remove "us-east-1b" from the middle:
variable "availability_zones" {
  default = ["us-east-1a", "us-east-1c"]   # removed 1b
}

# New plan:
# aws_subnet.private[0] → us-east-1a → no change ✅
# aws_subnet.private[1] → us-east-1c → WAS us-east-1b, NOW us-east-1c →
UPDATE ⚠
# aws_subnet.private[2] → GONE → DESTROY ❌

# Expected: destroy subnet-bbb only
# Actual: update subnet-bbb (becomes subnet-ccc) AND destroy old subnet-
ccc
# This is the INDEX SHIFT PROBLEM
```

🔵 When `count` Is Appropriate

```
✅ USE count when:
  - Creating N identical resources (same config, just different index)
  - Conditional resource creation (count = 0 or 1)
  - The set is static and items are never removed from the middle
  - Resources are truly interchangeable (any instance is the same)
  - Simple use case — don't over-engineer

❌ AVOID count when:
  - Each resource has different configuration
  - Items may be added/removed from the middle of the list
  - You need to reference resources by a meaningful name (not index)
  - Resources need to be independently managed
```

🔵 Short Interview Answer

> "`count` creates multiple instances of a resource using integer indexing. Each instance gets an address like `aws_instance.web[0]`, `aws_instance.web[1]`. The key use cases are creating N identical resources and conditional creation with `count = 0 or 1`. The critical limitation is index instability — if you remove an item from the middle of the list that drives count, Terraform shifts all subsequent indexes and updates or destroys resources you didn't intend to touch. This is why `for_each` is preferred for non-uniform resource sets — it uses stable string keys instead of shifting numeric indexes."

🔵 Real World Production Example

```
# Production pattern: count for identical worker nodes
resource "aws_instance" "worker" {
```

```
  count         = var.worker_count      # scale by changing this variable
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.worker_instance_type
  subnet_id     = element(aws_subnet.private[*].id, count.index %
length(aws_subnet.private))

  iam_instance_profile = aws_iam_instance_profile.worker.name

  user_data = templatefile("${path.module}/templates/worker-init.sh.tpl",
{
    worker_id   = count.index
    cluster_url = var.cluster_endpoint
  })

  tags = merge(local.common_tags, {
    Name   = "${local.prefix}-worker-${count.index}"
    Worker = "true"
    Index  = count.index
  })
}

# Attach all workers to load balancer
resource "aws_lb_target_group_attachment" "workers" {
  count            = var.worker_count
  target_group_arn = aws_lb_target_group.workers.arn
  target_id        = aws_instance.worker[count.index].id
  port             = var.worker_port
}
```

🔵 Common Interview Questions

Q: What is `count.index` and how is it used?

> "`count.index` is a built-in value available inside a resource block that uses `count`. It provides the current iteration's zero-based index — 0 for the first instance, 1 for the second, and so on. It's used to differentiate instances: assigning unique names (`web-${count.index}`), selecting different availability zones from a list (`var.azs[count.index]`), calculating different CIDR blocks (`cidrsubnet(var.cidr, 8, count.index)`), or referencing a matching resource from another count-based resource."

Q: What happens to count-based resources when you decrease the count?

> "Terraform destroys the highest-indexed instances. If you have `count = 3` (instances 0, 1, 2) and change to `count = 2`, Terraform destroys instance `[2]`. If you change to `count = 1`, it destroys `[1]` and `[2]`. This behavior is predictable when removing from the end, but removing from the middle (by changing the list that drives count) causes index shifting which can update or destroy unexpected resources."

🔵 Gotchas & Edge Cases

- ⚠ `count` **and** `for_each` **are mutually exclusive** — you can't use both on the same resource.
- ⚠ `count = 0` **doesn't delete existing state** — if you had `count = 3` and change to `count = 0`, Terraform plans to destroy all three instances. The resource block still exists in config, just with zero instances.
- `count` **value must be known at plan time** — if `count` depends on a computed value (like a resource ID), Terraform errors: "The count value depends on resource attributes that cannot be determined until apply." Use a known value (variable, local) for `count`.
- **Splat on count-based resources** — `aws_instance.web[*].id` returns a list. If `count = 0`, it returns an empty list `[]`, not null.
- `element()` **function for AZ distribution** — `element(list, index)` wraps around, useful for distributing across AZs: `element(var.azs, count.index)` cycles through AZs.

🔵 Connections to Other Concepts

- → **Topic 30 (`for_each`):** The solution to count's index instability problem
- → **Topic 31 (count vs for_each):** The critical comparison
- → **Category 11 (count to for_each migration):** How to migrate safely without destroying

---

Topic 30: ⚠ `for_each` — Map/Set Based, Why It's Better Than count

🔵 What It Is (Simple Terms)

`for_each` creates **multiple instances of a resource** using a **map or set** — each instance is identified by a **stable string key** instead of a numeric index. This solves count's fundamental index instability problem.

> ⚠ This is one of the most heavily tested intermediate Terraform topics. Interviewers probe the mechanics, the key/value access, and the superiority over count.

🔵 Why It Exists — What Problem It Solves

`for_each` solves the index shift problem of `count`. When you add or remove items from the collection, only the specific added/removed items change — all others are untouched.

```
count-based state (index-based):
  aws_subnet.private[0]   aws_subnet.private[1]   aws_subnet.private[2]
  Remove middle → shifts: [0] unchanged, [1] UPDATED, [2] DESTROYED

for_each-based state (key-based):
  aws_subnet.private["us-east-1a"]   aws_subnet.private["us-east-1b"]
aws_subnet.private["us-east-1c"]
  Remove "us-east-1b" → ONLY "us-east-1b" is destroyed, others untouched
```

🔵 `for_each` with a Set

```
# for_each with a set of strings — most common pattern
resource "aws_subnet" "private" {
  for_each = toset(["us-east-1a", "us-east-1b", "us-east-1c"])

  vpc_id            = aws_vpc.main.id
  availability_zone = each.key     # "us-east-1a", "us-east-1b", "us-east-
1c"
  cidr_block        = cidrsubnet(var.vpc_cidr, 8, index(tolist(toset(["us-
east-1a","us-east-1b","us-east-1c"])), each.key))

  tags = {
    Name = "private-${each.key}"
  }
}

# State entries created:
# aws_subnet.private["us-east-1a"]
# aws_subnet.private["us-east-1b"]
# aws_subnet.private["us-east-1c"]
```

🔵 `for_each` with a Map

```
# for_each with a map — access both key and value
variable "instances" {
  type = map(object({
    instance_type = string
    subnet_id     = string
  }))
  default = {
    web = { instance_type = "t3.medium", subnet_id = "subnet-aaa" }
    api = { instance_type = "t3.large",  subnet_id = "subnet-bbb" }
    worker = { instance_type = "t3.small", subnet_id = "subnet-ccc" }
  }
}

resource "aws_instance" "app" {
  for_each = var.instances

  ami           = data.aws_ami.ubuntu.id
  instance_type = each.value.instance_type   # "t3.medium", "t3.large",
"t3.small"
  subnet_id     = each.value.subnet_id

  tags = {
    Name = "${var.prefix}-${each.key}"       # "myapp-web", "myapp-api",
"myapp-worker"
```

```
    Role = each.key                          # "web", "api", "worker"
  }
}

# State entries:
# aws_instance.app["web"]    → i-aaa
# aws_instance.app["api"]    → i-bbb
# aws_instance.app["worker"] → i-ccc
```

🔵 `each.key` and `each.value`

```
resource "aws_iam_user" "team" {
  for_each = toset(var.usernames)
  # var.usernames = ["alice", "bob", "charlie"]

  name = each.key      # for sets: each.key = each.value = the set element
                       # for sets, key and value are the same thing
}

resource "aws_security_group_rule" "ingress" {
  for_each = {
    http  = 80
    https = 443
    ssh   = 22
  }

  type              = "ingress"
  security_group_id = aws_security_group.web.id
  from_port         = each.value    # 80, 443, 22
  to_port           = each.value
  protocol          = "tcp"
  cidr_blocks       = ["0.0.0.0/0"]

  description = "Allow ${each.key}"   # "Allow http", "Allow https",
"Allow ssh"
}
```

🔵 Accessing `for_each` Resources

```
# Reference a specific instance by key
output "web_instance_id" {
  value = aws_instance.app["web"].id
}

# Reference all instances — for_each returns a MAP not a list
output "all_instance_ids" {
  value = {
```

```
    for k, v in aws_instance.app : k => v.id
  }
  # { web = "i-aaa", api = "i-bbb", worker = "i-ccc" }
}

# Get just values as a list
output "instance_id_list" {
  value = values(aws_instance.app)[*].id
  # ["i-aaa", "i-bbb", "i-ccc"]  - ORDER NOT GUARANTEED
}

# Use in another for_each resource
resource "aws_eip" "app" {
  for_each = aws_instance.app    # reference another for_each resource
directly
  domain   = "vpc"
}

resource "aws_eip_association" "app" {
  for_each      = aws_instance.app
  instance_id   = each.value.id
  allocation_id = aws_eip.app[each.key].id
}
```

🔵 `for_each` with Complex Transformations

```
# Building a map from a list of objects for for_each
variable "subnets" {
  type = list(object({
    name = string
    cidr = string
    az   = string
    tier = string
  }))
  default = [
    { name = "web-a",  cidr = "10.0.1.0/24", az = "us-east-1a", tier =
"public"  },
    { name = "web-b",  cidr = "10.0.2.0/24", az = "us-east-1b", tier =
"public"  },
    { name = "app-a",  cidr = "10.0.3.0/24", az = "us-east-1a", tier =
"private" },
    { name = "app-b",  cidr = "10.0.4.0/24", az = "us-east-1b", tier =
"private" },
  ]
}

resource "aws_subnet" "main" {
  # Convert list to map keyed by name - for_each requires map or set, not
list
  for_each = { for s in var.subnets : s.name => s }
```

```
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value.cidr
  availability_zone = each.value.az

  tags = {
    Name = each.key              # "web-a", "web-b", "app-a", "app-b"
    Tier = each.value.tier       # "public" or "private"
  }
}

# State entries:
# aws_subnet.main["web-a"]
# aws_subnet.main["web-b"]
# aws_subnet.main["app-a"]
# aws_subnet.main["app-b"]
```

🔵 Why `for_each` Is Better Than `count` — The Stability Argument

```
Scenario: You have 3 IAM users managed with count
  aws_iam_user.team[0] → "alice"
  aws_iam_user.team[1] → "bob"
  aws_iam_user.team[2] → "charlie"

You need to remove "bob" (index 1):
  var.names = ["alice", "charlie"]   # removed "bob"

  count-based plan:
  ~ aws_iam_user.team[1]: name = "bob" -> "charlie"  (UPDATES alice's
account to charlie's name!)
  - aws_iam_user.team[2]: destroy                    (DESTROYS charlie's
account!)

  for_each-based plan:
  for_each = toset(["alice", "charlie"])  # removed "bob"
  - aws_iam_user.team["bob"]: destroy    (ONLY destroys bob — correct!)
  (alice and charlie: No changes)
```

🔵 Short Interview Answer

> "`for_each` creates resource instances identified by stable string keys from a map or set. Each instance's state address uses the key — `aws_instance.app[\"web\"]` — which is stable regardless of what other items are added or removed. This is fundamentally better than `count` because removing an item only affects that specific key's resource. With `count`, removing an item from the middle shifts all subsequent indexes, causing Terraform to update or destroy resources you didn't intend to touch. Inside the block, `each.key` gives the current key and `each.value` gives the corresponding value."

🔵 Common Interview Questions

Q: What types does `for_each` accept?

> "`for_each` accepts a map (any value type) or a set of strings. It does NOT accept a list directly — you must convert with `toset()` for a list of strings, or use a `for` expression to build a map from a list of objects: `{ for item in list : item.name => item }`. The reason for this constraint is that lists are ordered and indexable — Terraform wants keys, not positions."

Q: What is the difference between `each.key` and `each.value` for a set vs a map?

> "For a map: `each.key` is the map key (string) and `each.value` is the map value (any type — string, number, object). For a set of strings: `each.key` and `each.value` are both the same — the string element. This is because sets don't have separate keys and values; the element IS the key."

Q: Can you use `for_each` to reference another `for_each` resource?

> "Yes — and this is a powerful pattern. If resource B needs to mirror resource A's instances, you can write `for_each = resource_a.resource_name` to iterate over the same keys. For example, if you have `aws_instance.app` with `for_each`, you can create `aws_eip.app` with `for_each = aws_instance.app` and then use `each.key` to reference the matching instance. This ensures you always have exactly one EIP per instance, using the same key space."

🔵 Gotchas & Edge Cases

- ⚠ **`for_each` value must be known at plan time** — same as `count`. If the map values depend on computed attributes, you get "The 'for_each' value depends on resource attributes that cannot be determined until apply."
- ⚠ **`for_each` on a set loses order** — `values()` on a for_each resource returns instances in an undefined order. Don't rely on order for list-sensitive operations.
- **`for_each` on a list directly errors** — `for_each = var.list_of_strings` errors. Must use `toset(var.list_of_strings)` or a map.
- **Keys must be unique** — duplicate keys in the map error at plan time. If building a map from a list, ensure the key field is unique.
- **Removing a key destroys the resource** — there's no "rename" for `for_each` keys. Renaming a key = destroy old + create new. Use `moved` blocks to rename without recreation.

🔵 Connections to Other Concepts

- → **Topic 29 (`count`):** The alternative — understand both to explain tradeoffs
- → **Topic 31 (comparison):** The critical count vs for_each comparison
- → **Category 11 (count to for_each migration):** How to migrate between them
- → **Category 4 (Expressions):** `toset()`, `for` expressions used to prepare for_each input

---

Topic 31: ⚠ `count` vs `for_each` — The Critical Difference Every Interviewer Asks

---

🔵 What It Is (Simple Terms)

This is the single most-asked meta-argument question in Terraform interviews. Interviewers want to know if you understand the fundamental difference in how Terraform addresses and manages instances, and when each is appropriate.

> ⚠ Know this cold. Every mid-to-senior Terraform interview asks some version of this question.

🔵 The Head-to-Head Comparison

```
┌──────────────────────────────────────────────────────────────────┐
│                    count vs for_each                             │
│                                                                  │
│  ADDRESSING                                                      │
│  count:     resource[0], resource[1], resource[2]  (numeric index) │
│  for_each:  resource["key1"], resource["key2"]     (string key)  │
│                                                                  │
│  INPUT TYPE                                                      │
│  count:     integer (number)                                     │
│  for_each:  map or set of strings                                │
│                                                                  │
│  ITERATION VARIABLE                                              │
│  count:     count.index (0, 1, 2...)                             │
│  for_each:  each.key, each.value                                 │
│                                                                  │
│  STABILITY WHEN ITEMS REMOVED                                    │
│  count:     indexes shift → unintended updates/destroys          │
│  for_each:  only the removed key is destroyed                    │
│                                                                  │
│  WHEN ITEMS ADDED                                                │
│  count:     new index appended at end → safe                     │
│  for_each:  new key added → safe                                 │
│                                                                  │
│  WHEN ITEMS REMOVED FROM MIDDLE                                  │
│  count:     ⚠ ALL subsequent indexes shift → cascade updates     │
│  for_each:  ✅ Only the removed key is deleted → no cascade      │
│                                                                  │
│  RESOURCE IDENTITY                                               │
│  count:     identity = position in list                          │
│  for_each:  identity = meaningful string key                     │
└──────────────────────────────────────────────────────────────────┘
```

🔵 The Definitive Example — Why for_each Wins

```
# Scenario: Managing IAM users

# ── count approach ──────────────────────────────────────────────
variable "team" {
  default = ["alice", "bob", "charlie"]
}

resource "aws_iam_user" "team" {
  count = length(var.team)
  name  = var.team[count.index]
}

# State:
# aws_iam_user.team[0] → alice
# aws_iam_user.team[1] → bob
# aws_iam_user.team[2] → charlie

# Remove "bob":
variable "team" { default = ["alice", "charlie"] }

# Plan — THE PROBLEM:
# ~ aws_iam_user.team[1]: name = "bob" → "charlie"   ← RENAMES BOB TO
CHARLIE!
# - aws_iam_user.team[2]: destroy                    ← DESTROYS CHARLIE!
# This deletes charlie's user and renames bob's to charlie — WRONG

# ── for_each approach ───────────────────────────────────────────
resource "aws_iam_user" "team" {
  for_each = toset(var.team)
  name     = each.key
}

# State:
# aws_iam_user.team["alice"]   → alice
# aws_iam_user.team["bob"]     → bob
# aws_iam_user.team["charlie"] → charlie

# Remove "bob":
variable "team" { default = ["alice", "charlie"] }

# Plan — CORRECT:
# - aws_iam_user.team["bob"]: destroy   ← ONLY bob is deleted
# (alice and charlie: No changes)        ← untouched
```

🔵 When to Use Each

```
USE count WHEN:
  ✅ All instances are truly identical — same config, same purpose
  ✅ Conditional resource creation: count = var.enabled ? 1 : 0
```

```
    ✅ Simple scaling where index order doesn't matter
    ✅ Resources are fungible — any instance is interchangeable
    ✅ You'll only ever add/remove from the end of the list

    Examples:
    - 3 identical worker nodes (any worker can serve any request)
    - Conditional NAT gateway (either create or don't)
    - N identical read replicas

  USE for_each WHEN:
    ✅ Each instance has a distinct identity/purpose
    ✅ Instances may be added/removed individually
    ✅ You reference instances by name, not by position
    ✅ Instances have different configurations
    ✅ The key meaningfully identifies what the resource IS

    Examples:
    - IAM users (alice, bob, charlie — each is a different person)
    - Security group rules (http, https, ssh — each has different port)
    - Multi-environment deployments (dev, staging, prod)
    - Named server roles (web, api, db, worker)
```

🔵 Converting Between Them

```
# Converting a list to a set for for_each
variable "names" {
  type    = list(string)
  default = ["alice", "bob", "charlie"]
}

# ❌ Won't work — for_each doesn't accept list directly
for_each = var.names

# ✅ Convert to set
for_each = toset(var.names)

# Converting a list of objects to a map for for_each
variable "servers" {
  type = list(object({ name = string, type = string }))
  default = [
    { name = "web",    type = "t3.medium" },
    { name = "api",    type = "t3.large"  },
    { name = "worker", type = "t3.small"  },
  ]
}

# ❌ Won't work — for_each doesn't accept list
for_each = var.servers

# ✅ Convert to map keyed by name
```

```
for_each = { for s in var.servers : s.name => s }
# Result: { "web" = {...}, "api" = {...}, "worker" = {...} }
```

🔵 The `count` = 0 or 1 Pattern vs `for_each` = {} or {key=val}

```
# count = 0/1 is idiomatic for optional resources
resource "aws_eip" "nat" {
  count  = var.create_nat ? 1 : 0
  domain = "vpc"
}

# for_each = {} for optional is less clean but equivalent
resource "aws_eip" "nat" {
  for_each = var.create_nat ? { "nat" = true } : {}
  domain   = "vpc"
}

# Verdict: count is better for simple optional (0 or 1)
# for_each is better when you need a meaningful key
```

🔵 Performance Comparison

```
count:
  - State addresses are compact: resource[0]
  - Slightly less overhead for large identical resource sets
  - Plan shows index numbers (less readable for named resources)

for_each:
  - State addresses are readable: resource["web"]
  - Slightly more overhead due to key-value structure
  - Plan shows meaningful names (much more readable)
  - Better for debugging: "aws_instance.app[\"web\"] will be updated"
    vs "aws_instance.app[2] will be updated" (what is index 2?)
```

🔵 Short Interview Answer

> "The core difference is addressing: `count` uses numeric indexes (`resource[0]`) while `for_each` uses stable string keys (`resource[\"name\"]`). This makes `for_each` fundamentally safer — removing an item only destroys that specific key. With `count`, removing an item from the middle shifts all subsequent indexes, causing Terraform to update resources you didn't intend to touch. Use `count` for truly identical resources and conditional creation (0 or 1). Use `for_each` when instances have distinct identities, different configurations, or may be individually added/removed. The classic example that demonstrates the difference is managing IAM users — with `count`, removing a user

> from the middle renames the next user and deletes the last. With `for_each`, only the removed user is deleted."

🔵 Common Interview Questions

Q: Can you use both `count` and `for_each` on the same resource?

> "No — they're mutually exclusive. Using both causes an error: 'The arguments "count" and "for_each" are both defined.' You must choose one. If you need to conditionally create multiple different resources, use `for_each` with an empty map (`{}`) for zero instances or a populated map for instances."

Q: How does Terraform handle `for_each` when the map has a computed value as a key?

> "If any key in the `for_each` map is a computed value (known only after apply), Terraform errors with 'The for_each value depends on resource attributes that cannot be determined until apply.' Keys must be known at plan time. This is a common pain point — workaround is to use a variable or local for the key rather than a computed resource attribute."

Q: A colleague says 'just always use `for_each`, never `count`'. Do you agree?

> "Mostly, but not entirely. `for_each` is strictly better when instances have distinct identities. But `count` is still appropriate and cleaner for two cases: conditional creation (`count = var.enabled ? 1 : 0`) — this is more readable than `for_each = var.enabled ? {\"this\" = true} : {}`; and truly identical fungible resources where you just need N of them and don't care which is which. For everything else — any time instances have different configs or may be individually removed — `for_each` is correct."

🔵 Gotchas & Edge Cases

- ⚠ **Migrating from `count` to `for_each` destroys and recreates** — see Category 11 Topic 98 for the safe migration path using `terraform state mv`.
- ⚠ **`for_each` on the result of a `for` expression** — if you do `for_each = { for item in list : item.id => item }` and `item.id` is computed, the whole thing fails at plan time.
- **`values()` function for for_each output** — `values(aws_instance.app)` returns a list of resource instances in undefined order. Don't use index-based access on this result.
- **Empty `for_each`** — `for_each = {}` or `for_each = toset([])` creates zero instances. This is the `for_each` equivalent of `count = 0`.

🔵 Connections to Other Concepts

- → **Topic 29 (`count`):** The full count explanation
- → **Topic 30 (`for_each`):** The full for_each explanation
- → **Topic 34 (`replace_triggered_by`):** Works with both count and for_each
- → **Category 11 Topic 98:** Migrating between them safely

Topic 32: `lifecycle` Block — `create_before_destroy`, `prevent_destroy`, `ignore_changes`

🔵 What It Is (Simple Terms)

The `lifecycle` block is a meta-argument that customizes **how Terraform manages the creation, update, and destruction** of a resource. It overrides Terraform's default behavior for specific resources where the defaults are wrong for your use case.

🔵 Why It Exists — What Problem It Solves

Terraform's default behavior is:

- Replace = destroy old first, then create new (causes downtime)
- Any resource can be destroyed by `terraform destroy` or config removal
- All config changes are applied (even ones you want to ignore)

`lifecycle` lets you override each of these defaults per resource.

🔵 Full `lifecycle` Block

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"

  lifecycle {
    # ── create_before_destroy ──────────────────────────────────────
    create_before_destroy = true

    # ── prevent_destroy ────────────────────────────────────────────
    prevent_destroy = true

    # ── ignore_changes ─────────────────────────────────────────────
    ignore_changes = [
      ami,
      tags["LastUpdated"],
    ]

    # ── replace_triggered_by (Terraform 1.2+) ──────────────────────
    replace_triggered_by = [
      aws_launch_template.web.id
    ]
  }
}
```

🔵 `create_before_destroy` — Zero-Downtime Replacement

```
# DEFAULT behavior (create_before_destroy = false):
# 1. Destroy old resource  ← DOWNTIME HERE
# 2. Create new resource

# WITH create_before_destroy = true:
# 1. Create new resource  ← no downtime (both exist briefly)
# 2. Destroy old resource

resource "aws_security_group" "web" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id

  lifecycle {
    create_before_destroy = true
    # CRITICAL for security groups attached to running instances
    # Without this: destroy old SG → EC2 loses network access → create new
SG
    # With this: create new SG → attach to EC2 → destroy old SG
  }
}

resource "aws_lb" "web" {
  name               = "web-lb"
  internal           = false
  load_balancer_type = "application"

  lifecycle {
    create_before_destroy = true
    # Ensures new LB is ready before old one is removed
  }
}
```

The Constraint Propagation Problem

```
# ⚠ create_before_destroy propagates through dependencies
# If resource A has create_before_destroy = true
# AND resource B depends on A
# Then B must ALSO support create_before_destroy
# Otherwise Terraform errors with a dependency cycle

resource "aws_security_group" "web" {
  lifecycle { create_before_destroy = true }
}

resource "aws_instance" "web" {
  vpc_security_group_ids = [aws_security_group.web.id]
  # aws_instance.web implicitly gets create_before_destroy = true
```

```
  # because its dependency (SG) has it
  # This is usually fine — but know that it propagates
}
```

🔵 `prevent_destroy` — Protecting Critical Resources

```
# Terraform ERRORS if any plan would destroy this resource
resource "aws_db_instance" "production" {
  identifier        = "prod-database"
  instance_class    = "db.r5.large"
  engine            = "postgres"
  engine_version    = "15.4"
  allocated_storage = 500

  lifecycle {
    prevent_destroy = true
    # Error you'll see if destroy is attempted:
    # Error: Instance cannot be destroyed
    # Resource aws_db_instance.production has lifecycle.prevent_destroy
    # set, but the plan calls for this resource to be destroyed.
    # To avoid this error and continue, you must remove the
    # prevent_destroy attribute and run a new plan and apply.
  }
}

# Common candidates for prevent_destroy:
resource "aws_s3_bucket" "terraform_state" {
  lifecycle { prevent_destroy = true }
}

resource "aws_dynamodb_table" "terraform_locks" {
  lifecycle { prevent_destroy = true }
}

resource "aws_kms_key" "production" {
  lifecycle { prevent_destroy = true }
}
```

What `prevent_destroy` Does NOT Protect Against

```
# prevent_destroy only works within Terraform
# It does NOT protect against:
# - Manual deletion via AWS Console
# - AWS CLI delete commands
# - Other Terraform configs that manage the same resource
# - terraform state rm followed by manual deletion

# To protect against manual deletion, use AWS-level protection:
```

```
resource "aws_db_instance" "production" {
  deletion_protection = true    # AWS refuses DELETE API calls
  lifecycle { prevent_destroy = true }  # Terraform refuses to plan
deletion
}
```

🔵 `ignore_changes` — Preventing Config Drift Reversion

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"
  user_data     = file("scripts/init.sh")

  lifecycle {
    ignore_changes = [
      # Ignore AMI changes — don't replace instance when new AMI is
released
      ami,

      # Ignore specific tag key (other tags are still managed)
      tags["LastDeployed"],
      tags["PatchedAt"],

      # Ignore user_data — instance is already initialized, don't replace
      user_data,
    ]
  }
}

# ignore_changes = all — ignore EVERY attribute after creation
resource "aws_ecs_service" "app" {
  name            = "app-service"
  cluster         = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count   = 3

  lifecycle {
    # ECS desired_count is managed by auto-scaling, not Terraform
    # ignore_changes = all means Terraform never updates this resource
    ignore_changes = [desired_count, task_definition]
  }
}
```

Common `ignore_changes` Use Cases

```
# 1. ASG desired capacity managed by auto-scaling
resource "aws_autoscaling_group" "web" {
  lifecycle {
```

```
    ignore_changes = [desired_capacity]
  }
}

# 2. ECS service managed by deployment pipelines
resource "aws_ecs_service" "app" {
  lifecycle {
    ignore_changes = [task_definition, desired_count]
  }
}

# 3. Timestamps in tags
resource "aws_s3_bucket" "data" {
  tags = { LastModified = timestamp() }
  lifecycle {
    ignore_changes = [tags["LastModified"]]
  }
}

# 4. Security group rules managed externally
resource "aws_security_group" "managed_externally" {
  lifecycle {
    ignore_changes = [ingress, egress]
  }
}
```

🔵 Combining Multiple Lifecycle Settings

```
# Production database - full lifecycle protection
resource "aws_db_instance" "production" {
  identifier              = "prod-db"
  instance_class          = "db.r5.xlarge"
  engine                  = "postgres"
  engine_version          = "15.4"
  allocated_storage       = 1000
  multi_az                = true
  deletion_protection     = true   # AWS-level protection

  lifecycle {
    prevent_destroy       = true   # Terraform-level protection

    create_before_destroy = true   # Zero downtime for updates that
require replacement

    ignore_changes = [
      password,           # Password rotated externally via Secrets
Manager
      snapshot_identifier # Restored from snapshot initially - ignore
after
    ]
```

```
    }
  }
```

🔵 Short Interview Answer

> "The `lifecycle` block has four settings. `create_before_destroy = true` reverses the default replacement order — creates the new resource first, then destroys the old one, enabling zero-downtime replacements. `prevent_destroy = true` causes Terraform to error if any plan would destroy the resource — protecting production databases, state buckets, KMS keys. `ignore_changes` tells Terraform to ignore drift on specific attributes after initial creation — used for attributes managed by external systems like ASG desired count or ECS task definition. And `replace_triggered_by` forces replacement when a referenced resource changes. The gotcha with `create_before_destroy` is that it propagates through the dependency graph — all dependent resources must also support concurrent existence of old and new versions."

🔵 Common Interview Questions

Q: What's the difference between `prevent_destroy = true` and AWS `deletion_protection = true`?

> "`prevent_destroy` is a Terraform guard — it prevents the Terraform plan from including a destroy operation for this resource. If you try to run `terraform destroy` or remove the resource from config, Terraform errors before making any API calls. It does nothing to prevent manual AWS Console or CLI deletions. `deletion_protection` is an AWS API flag — it makes the AWS API refuse any delete request, regardless of who sends it. Best practice is both: `prevent_destroy` catches Terraform mistakes, `deletion_protection` catches all other mistakes."

Q: You set `ignore_changes = [ami]` on an EC2 instance. A security vulnerability is found in the current AMI. How do you force the update?

> "Remove `ignore_changes = [ami]` from the lifecycle block, update the AMI reference in config, run `terraform plan` — it should now show the instance needs replacement. Apply it. Then add `ignore_changes = [ami]` back to prevent future AMI changes from triggering replacements. Alternatively, use `terraform taint` (deprecated) or `terraform apply -replace=aws_instance.web` to force replacement once without removing the ignore_changes permanently."

🔵 Gotchas & Edge Cases

- ⚠ `prevent_destroy` **doesn't work if you remove the resource from config** — if you delete the resource block entirely AND the lifecycle block with it, Terraform no longer sees `prevent_destroy` and proceeds with destruction. The protection only works when the resource block exists in config.
- ⚠ `ignore_changes = all` **means Terraform never updates the resource** — useful for externally-managed resources but dangerous if you want Terraform to be the source of truth. Changes made in Terraform config are silently ignored after creation.

- `create_before_destroy` **requires unique names** — if the old and new resources can't coexist (e.g., both use the same name), you need to generate unique names using `random_id` or `timestamp`. AWS doesn't allow two resources with the same name to exist simultaneously.
- `ignore_changes` **on nested blocks** — you can ignore entire nested blocks or specific attributes within them: `ignore_changes = [ingress]` ignores all ingress rules.

🔵 Connections to Other Concepts

- → **Topic 16 (Replacement):** `lifecycle` controls what triggers and how replacement happens
- → **Topic 34 (`replace_triggered_by`):** The fourth lifecycle setting
- → **Category 8 (Security):** `prevent_destroy` is a security/safety control
- → **Category 11 (Troubleshooting):** `ignore_changes` causes silent drift (Topic 93)

Topic 33: `provider` Meta-Argument — Cross-Account, Cross-Region Patterns

---

🔵 What It Is (Simple Terms)

The `provider` meta-argument on a resource block explicitly selects **which provider instance** should manage that resource. By default, resources use the default (non-aliased) provider. The `provider` meta-argument overrides this to use a specific aliased instance.

🔵 Why It Exists — What Problem It Solves

Without the `provider` meta-argument:

- All resources in a module use the same provider configuration
- You can't create resources in multiple regions within one config
- You can't manage resources across multiple AWS accounts
- Global resources (like ACM certs for CloudFront) that must be in `us-east-1` can't coexist with regional resources

---

🔵 Basic Syntax

```
# Define provider instances
provider "aws" {
  region = "eu-west-1"          # default provider — no alias
}

provider "aws" {
  alias  = "us_east_1"
  region = "us-east-1"
```

```
}

provider "aws" {
  alias  = "prod_account"
  region = "eu-west-1"
  assume_role {
    role_arn = "arn:aws:iam::999999999999:role/TerraformRole"
  }
}

# Use specific provider instance on a resource
resource "aws_acm_certificate" "cloudfront" {
  provider          = aws.us_east_1    # syntax: <provider_type>.<alias>
  domain_name       = "example.com"
  validation_method = "DNS"
}

resource "aws_vpc" "prod" {
  provider   = aws.prod_account        # creates VPC in prod account
  cidr_block = "10.0.0.0/16"
}
```

🔵 Cross-Region Pattern — ACM + CloudFront

```
# ── Providers ──────────────────────────────────────────────────────
provider "aws" {
  region = var.primary_region     # e.g., eu-west-1
}

provider "aws" {
  alias  = "us_east_1"
  region = "us-east-1"            # required for CloudFront ACM
}

# ── ACM Certificate (MUST be us-east-1 for CloudFront) ────────────────
resource "aws_acm_certificate" "cdn" {
  provider          = aws.us_east_1
  domain_name       = "cdn.example.com"
  validation_method = "DNS"

  lifecycle {
    create_before_destroy = true
  }
}

# ── DNS Validation (Route53 is global — default provider fine) ────────
resource "aws_route53_record" "cert_validation" {
  for_each = {
    for dvo in aws_acm_certificate.cdn.domain_validation_options :
    dvo.domain_name => {
```

```
      name   = dvo.resource_record_name
      record = dvo.resource_record_value
      type   = dvo.resource_record_type
    }
  }

  zone_id = data.aws_route53_zone.main.zone_id
  name    = each.value.name
  type    = each.value.type
  records = [each.value.record]
  ttl     = 60
}

resource "aws_acm_certificate_validation" "cdn" {
  provider                = aws.us_east_1
  certificate_arn         = aws_acm_certificate.cdn.arn
  validation_record_fqdns = [for record in
aws_route53_record.cert_validation : record.fqdn]
}

# ── CloudFront Distribution (global — default provider) ────────────────
resource "aws_cloudfront_distribution" "cdn" {
  # default provider
  viewer_certificate {
    acm_certificate_arn      = aws_acm_certificate.cdn.arn   # us-east-1
cert
    ssl_support_method       = "sni-only"
    minimum_protocol_version = "TLSv1.2_2021"
  }
  # ...
}
```

🔵 Cross-Account Pattern

```
# ── Provider setup for multi-account ──────────────────────────────────
# Base identity (CI/CD runner has this role in the tools account)
provider "aws" {
  region = "eu-west-1"
  assume_role {
    role_arn = "arn:aws:iam::111111111111:role/TerraformBase"
  }
}

# Dev account
provider "aws" {
  alias  = "dev"
  region = "eu-west-1"
  assume_role {
    role_arn = "arn:aws:iam::222222222222:role/TerraformRole"
  }
```

```
}

# Prod account
provider "aws" {
  alias  = "prod"
  region = "eu-west-1"
  assume_role {
    role_arn = "arn:aws:iam::333333333333:role/TerraformRole"
  }
}

# ── Resources in specific accounts ──────────────────────────────
resource "aws_vpc" "dev_network" {
  provider   = aws.dev
  cidr_block = "10.1.0.0/16"
  tags = { Name = "dev-vpc", Account = "dev" }
}

resource "aws_vpc" "prod_network" {
  provider   = aws.prod
  cidr_block = "10.0.0.0/16"
  tags = { Name = "prod-vpc", Account = "prod" }
}

# ── VPC peering across accounts ──────────────────────────────────
resource "aws_vpc_peering_connection" "dev_to_prod" {
  provider    = aws.dev              # peering REQUEST from dev
  vpc_id      = aws_vpc.dev_network.id
  peer_vpc_id = aws_vpc.prod_network.id
  peer_owner_id = "333333333333"     # prod account ID
  auto_accept = false
}

resource "aws_vpc_peering_connection_accepter" "dev_to_prod" {
  provider                  = aws.prod    # peering ACCEPT in prod
  vpc_peering_connection_id = aws_vpc_peering_connection.dev_to_prod.id
  auto_accept               = true
}
```

🔵 `provider` Meta-Argument in Modules

```
# Passing providers to modules — required for aliased providers
module "cdn" {
  source = "./modules/cdn"

  providers = {
    aws           = aws              # default provider → module's default
    aws.us_east_1 = aws.us_east_1   # aliased → module's aliased
  }
}
```

```
# Module must declare expected aliases:
# modules/cdn/versions.tf
terraform {
  required_providers {
    aws = {
      source                = "hashicorp/aws"
      version               = "~> 5.0"
      configuration_aliases = [aws.us_east_1]   # declares alias
requirement
    }
  }
}
```

🔵 Short Interview Answer

> "The `provider` meta-argument on a resource selects which provider instance manages it — syntax is `provider = aws.<alias>`. Without it, resources use the default non-aliased provider. It's essential for cross-region scenarios like ACM certificates that must be in `us-east-1` for CloudFront while all other resources are in `eu-west-1`, and for cross-account patterns where you assume different IAM roles per account using provider aliases. In modules, aliased providers must be explicitly passed via the `providers` map argument — modules don't inherit aliases automatically."

🔵 Gotchas & Edge Cases

- ⚠ **`provider` meta-argument syntax** — it's `provider = aws.alias_name`, NOT `provider = "aws.alias_name"` (no quotes around the reference).
- ⚠ **Modules don't inherit aliases** — the most common mistake. Define the alias in root, forget to pass it to the module → module silently uses the default provider (wrong region/account).
- **Each aliased provider = separate process** — with 5 provider aliases you have 5 provider subprocesses. Minor performance consideration in large configs.
- **Removing an alias renames in state** — if you rename a provider alias, Terraform may think resources need to move to the new provider, causing unexpected replacements.

🔵 Connections to Other Concepts

- → **Category 2 Topic 10 (Aliases):** Full provider alias coverage
- → **Category 2 Topic 8 (Auth):** Each aliased provider has its own auth config
- → **Category 10 (Multi-account patterns):** Provider aliases are the foundation

---

Topic 34: ⚠ `replace_triggered_by` — When and Why (Terraform 1.2+)

🔵 What It Is (Simple Terms)

`replace_triggered_by` is a lifecycle setting that forces a resource to be **replaced (destroyed and recreated)** whenever a referenced resource or attribute changes — even if the current resource's own config hasn't changed.

> ⚠ This is a newer feature (Terraform 1.2+) that interviewers use to test if candidates are current with Terraform's evolution. Understanding when it solves a real problem distinguishes strong candidates.

🔵 Why It Exists — What Problem It Solves

Before `replace_triggered_by`, there was no clean way to say "replace resource B whenever resource A changes." Your options were:

1. **Manual `terraform taint`** — imperative, not codified
2. **Hack with `null_resource` and triggers** — workaround, not clean
3. **Accept stale config** — resource B runs with outdated config from A

`replace_triggered_by` codifies this dependency in the lifecycle block declaratively.

🔵 Syntax

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"
  user_data     = templatefile("${path.module}/scripts/init.sh.tpl", {
    config_version = aws_launch_template.web.latest_version
  })

  lifecycle {
    replace_triggered_by = [
      aws_launch_template.web.id,      # replace when launch template ID
changes
      aws_launch_template.web          # OR: reference entire resource
(any change)
    ]
  }
}
```

🔵 Real World Use Cases

Use Case 1: Rolling Instance Replacement When Launch Template Changes

```
resource "aws_launch_template" "web" {
  name_prefix   = "web-lt-"
  image_id      = data.aws_ami.ubuntu.id
```

```
  instance_type = "t3.medium"

  user_data =
base64encode(templatefile("${path.module}/scripts/init.sh.tpl", {
    app_version = var.app_version
    config_hash = sha256(file("${path.module}/config/app.conf"))
  }))

  # When user_data changes (new app version, config change),
  # a new launch template version is created
  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_autoscaling_group" "web" {
  name = "web-asg-${aws_launch_template.web.latest_version}"
  # The ASG name includes the LT version — this forces recreation
  # when the launch template changes, triggering a rolling replacement

  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }

  lifecycle {
    replace_triggered_by = [
      aws_launch_template.web   # Recreate ASG when launch template
changes
    ]
    create_before_destroy = true
  }
}
```

Use Case 2: Re-Running Initialization When Config Changes

```
resource "aws_s3_object" "app_config" {
  bucket  = aws_s3_bucket.config.id
  key     = "app.conf"
  content = templatefile("${path.module}/templates/app.conf.tpl",
var.app_config)
  etag    = md5(templatefile("${path.module}/templates/app.conf.tpl",
var.app_config))
}

resource "aws_instance" "app" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"

  # Instance doesn't directly reference the config object
  # But it needs to be replaced when config changes
```

```
  # (re-running init script to pick up new config)

  lifecycle {
    replace_triggered_by = [
      aws_s3_object.app_config.etag   # replace when config content
changes
    ]
  }
}
```

Use Case 3: Rotating Certificates

```
resource "aws_acm_certificate" "app" {
  domain_name       = "app.example.com"
  validation_method = "DNS"

  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_lb_listener" "https" {
  load_balancer_arn = aws_lb.app.arn
  port              = 443
  protocol          = "HTTPS"
  certificate_arn   = aws_acm_certificate.app.arn

  # When certificate is rotated (new ACM certificate created),
  # force listener replacement to pick up the new cert
  lifecycle {
    replace_triggered_by = [
      aws_acm_certificate.app.id
    ]
  }
}
```

🔵 `replace_triggered_by` vs `depends_on`

```
depends_on:
  → Controls ORDERING — ensures resource B is applied AFTER resource A
  → Does NOT force replacement of B when A changes
  → B is only replaced if its OWN config changes

replace_triggered_by:
  → Forces REPLACEMENT of resource B when resource A changes
  → Even if B's own config is identical
  → Used when B needs to be recreated to pick up changes in A
```

🔵 `replace_triggered_by` vs `triggers` in `null_resource`

```
# ❌ Old way - null_resource with triggers (hacky workaround)
resource "null_resource" "reboot_on_config_change" {
  triggers = {
    config_hash = sha256(file("config.json"))
  }

  provisioner "local-exec" {
    command = "aws ec2 reboot-instances --instance-ids
${aws_instance.web.id}"
  }
}

# ✅ New way - replace_triggered_by (clean, declarative)
resource "aws_instance" "web" {
  lifecycle {
    replace_triggered_by = [
      # Reference a resource whose change should trigger replacement
      aws_s3_object.config.etag
    ]
  }
}
```

🔵 What Triggers a `replace_triggered_by`

```
lifecycle {
  replace_triggered_by = [
    # Reference to entire resource - any change to resource triggers
replacement
    aws_launch_template.web,

    # Reference to specific attribute - only changes to this attribute
trigger
    aws_launch_template.web.id,
    aws_launch_template.web.latest_version,

    # Reference to count-indexed resource
    aws_instance.worker[0],

    # Reference to for_each resource
    aws_instance.worker["web"],

    # Can mix multiple triggers
    aws_launch_template.web.id,
    aws_s3_object.config.etag,
  ]
}
```

🔵 Short Interview Answer

> "`replace_triggered_by` is a lifecycle setting introduced in Terraform 1.2 that forces a resource to be replaced whenever a referenced resource or attribute changes — even if the resource's own configuration hasn't changed. The canonical use case is forcing an EC2 instance or ASG to be recreated when a launch template changes, ensuring the new instances use the updated template. Before this feature, engineers used the `null_resource` with `triggers` as an ugly workaround. The difference from `depends_on` is important: `depends_on` controls ordering, `replace_triggered_by` forces replacement — very different behaviors."

🔵 Real World Production Example

```
# Production pattern: ensure EKS nodes are always running
# the latest approved AMI and bootstrap config

data "aws_ami" "eks_node" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["amazon-eks-node-${var.eks_version}-*"]
  }
}

resource "aws_launch_template" "eks_nodes" {
  name_prefix   = "eks-nodes-"
  image_id      = data.aws_ami.eks_node.id

  user_data = base64encode(templatefile(
    "${path.module}/templates/eks-bootstrap.sh.tpl",
    {
      cluster_name    = aws_eks_cluster.main.name
      cluster_ca      = aws_eks_cluster.main.certificate_authority[0].data
      cluster_endpoint = aws_eks_cluster.main.endpoint
    }
  ))

  lifecycle { create_before_destroy = true }
}

resource "aws_eks_node_group" "workers" {
  cluster_name    = aws_eks_cluster.main.name
  node_group_name =
"workers-${aws_launch_template.eks_nodes.latest_version}"
  node_role_arn   = aws_iam_role.eks_nodes.arn
  subnet_ids      = aws_subnet.private[*].id
```

```
    launch_template {
      id      = aws_launch_template.eks_nodes.id
      version = aws_launch_template.eks_nodes.latest_version
    }

    lifecycle {
      # Force node group replacement when launch template changes
      # Ensures all nodes are drained and replaced with new AMI/config
      replace_triggered_by = [aws_launch_template.eks_nodes]
      create_before_destroy = true
    }
  }
```

🔵 Common Interview Questions

Q: What is `replace_triggered_by` and when would you use it?

> "`replace_triggered_by` forces a resource to be replaced (destroyed and recreated) when any referenced resource or attribute changes — even if the resource itself hasn't changed. Use it when a resource needs to be recreated to pick up changes in a dependency that doesn't create an automatic update. The canonical example is an ASG or EC2 instance that references a launch template — if the launch template changes, Terraform updates the template but the running instances don't automatically restart. Adding `replace_triggered_by = [aws_launch_template.web]` ensures the ASG is replaced (triggering a rolling node update) whenever the template changes."

Q: How is `replace_triggered_by` different from just making the resource reference the changing attribute directly?

> "If resource B directly references attribute `X` of resource A, Terraform may update B in-place when X changes — that's an update, not a replace. `replace_triggered_by` explicitly forces a REPLACE (destroy + create), not an in-place update. This is important when the resource needs to be fully recreated to pick up the change — like an EC2 instance that ran its init script on creation and won't re-run it on an in-place update."

🔵 Gotchas & Edge Cases

- ⚠ **`replace_triggered_by` requires Terraform 1.2+** — older versions will error on this argument. Make sure `required_version` enforces 1.2+.
- ⚠ **Replacement can cascade** — if resource B is replaced and resource C depends on B's ID, C may also need replacement. Always review the full plan.
- **`replace_triggered_by` ignores `ignore_changes`** — even if an attribute is in `ignore_changes`, `replace_triggered_by` can still trigger replacement based on the referenced resource's changes.
- **Reference must be to a managed resource** — you can't reference data sources or provider-defined functions in `replace_triggered_by`.
- **Combining with `create_before_destroy`** — almost always use both together for production resources to minimize downtime during replacement.

🔵 Connections to Other Concepts

- → **Topic 32 (`lifecycle`):** `replace_triggered_by` is part of the lifecycle block
- → **Topic 16 (Replacement):** Understanding what triggers replacement generally
- → **Topic 15 (`depends_on`):** Key difference — ordering vs replacement forcing
- → **Category 9 (CI/CD):** Rolling deployments via `replace_triggered_by` + `create_before_destroy`

📊 Category 5 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight
29. `count`	Integer-indexed instances, limitations with index shifting	⭐⭐⭐⭐
30. `for_each` ⚠	Stable key-based instances, each.key/each.value	⭐⭐⭐⭐⭐
31. count vs for_each ⚠	Index shift = count's fatal flaw, for_each uses stable keys	⭐⭐⭐⭐⭐
32. `lifecycle`	create_before_destroy, prevent_destroy, ignore_changes	⭐⭐⭐⭐⭐
33. `provider` meta-arg	Cross-region/account, aliases, module provider passing	⭐⭐⭐⭐
34. `replace_triggered_by` ⚠	Force replacement on external change — TF 1.2+	⭐⭐⭐⭐

🔑 Category 5 — Critical Rules

```
count rules:
  count.index is 0-based                          |  count = 0 means zero
instances
  count must be known at plan time                |  count and for_each
are mutually exclusive
  Removing from middle = index shift = danger     |

for_each rules:
  Accepts map or set only — NOT list              |  Convert list →
toset() or for expression
  each.key = each.value for sets                  |  each.key ≠ each.value
for maps
  Keys must be unique                             |  Keys must be known at
plan time
  for_each = {} means zero instances             |
```

```
lifecycle rules:
  create_before_destroy propagates through deps   |  prevent_destroy needs
resource block to exist
  ignore_changes = all = Terraform never updates  |  ignore_changes causes
silent drift
  prevent_destroy ≠ AWS deletion_protection       |  Use both for full
protection

replace_triggered_by:
  Terraform 1.2+ only                             |  Forces REPLACE not
just update
  Different from depends_on (ordering vs replace) |  Combine with
create_before_destroy
```

🎯 Category 5 — Top 5 Interview Questions to Master

1. **"What is the difference between `count` and `for_each`? When would you use each?"** — index shift problem, stable keys, identity vs fungibility
2. **"I have 3 IAM users managed with `count`. If I remove the middle one, what happens?"** — indexes shift, wrong users get updated/deleted — the definitive count problem
3. **"What does `create_before_destroy` do and what constraint does it impose?"** — zero-downtime replacement, dependency propagation
4. **"How do you prevent a production database from being accidentally destroyed by Terraform?"** — `prevent_destroy` + AWS `deletion_protection`, why you need both
5. **"What is `replace_triggered_by` and how is it different from `depends_on`?"** — ordering vs replacement forcing, launch template use case

---

> **Next:** Category 6 — State Management & Workspaces (Topics 35–46) Type `Category 6` to continue, `quiz me` to be tested on Category 5, or `deeper` on any specific topic.